

Quaternion Math for a CubeSat ADCS Digital Twin

What the Math Does and Why Each Piece Exists

1 The core problem

A CubeSat has an orientation in three-dimensional space, and the flight computer needs a mathematical way to represent, update, compare, and transform that orientation.

We use two coordinate frames:

- **Inertial frame I :** a non-rotating reference frame used by the orbital/environment model.
- **Body frame B :** a coordinate frame rigidly attached to the spacecraft.

The central attitude-determination and control question is:

How is the spacecraft body frame oriented relative to the inertial frame?

For this project we represent that orientation with a scalar-first Hamilton quaternion

$$q_{IB} = [q_w \quad q_x \quad q_y \quad q_z]^T,$$

where q_{IB} rotates a vector expressed in the body frame into the inertial frame.

2 Why quaternions instead of roll, pitch, and yaw?

Euler angles are intuitive, but they have singularities. At certain orientations two rotational axes align, producing the classic gimbal-lock problem. A spacecraft may need to pass through arbitrary orientations, so an internal attitude representation should not have this failure mode.

A quaternion has four components,

$$q = [q_w \quad q_x \quad q_y \quad q_z]^T,$$

with the unit-length constraint

$$q_w^2 + q_x^2 + q_y^2 + q_z^2 = 1.$$

The constraint removes one independent degree of freedom, so the quaternion still represents the three physical rotational degrees of freedom.

Quaternions are attractive for spacecraft ADCS because they provide:

- no gimbal-lock singularity,
- efficient composition of rotations,

- convenient attitude-error calculations,
- numerically stable propagation,
- clean integration with estimation and control algorithms.

3 What a quaternion physically represents

A rotation through angle θ about unit axis

$$\hat{e} = [e_x \quad e_y \quad e_z]^T$$

can be represented by

$$q = \begin{bmatrix} \cos(\theta/2) \\ e_x \sin(\theta/2) \\ e_y \sin(\theta/2) \\ e_z \sin(\theta/2) \end{bmatrix}.$$

Thus it is often useful to view a quaternion as a scalar plus a three-vector:

$$q = \begin{bmatrix} s \\ \mathbf{v} \end{bmatrix}.$$

For example, a 90° rotation about $+Z$ is

$$q = \begin{bmatrix} \cos 45^\circ \\ 0 \\ 0 \\ \sin 45^\circ \end{bmatrix} \approx \begin{bmatrix} 0.7071 \\ 0 \\ 0 \\ 0.7071 \end{bmatrix}.$$

4 Why quatMultiply exists

Rotations cannot be combined by ordinary vector addition. If one rotation is represented by q_1 and another by q_2 , the combined rotation is obtained with the Hamilton product

$$q = q_1 \otimes q_2.$$

Writing

$$q_1 = \begin{bmatrix} s_1 \\ v_1 \end{bmatrix}, \quad q_2 = \begin{bmatrix} s_2 \\ v_2 \end{bmatrix},$$

the product is

$$q_1 \otimes q_2 = \begin{bmatrix} s_1 s_2 - v_1^T v_2 \\ s_1 v_2 + s_2 v_1 + v_1 \times v_2 \end{bmatrix}.$$

The dot product contributes to the scalar portion, while the cross product captures the three-dimensional interaction between the two rotations.

This operation later becomes central to control. If the spacecraft currently has attitude q and the desired attitude is q_d , an attitude-error quaternion can be formed from a product involving the desired attitude and the inverse of the current attitude, for example

$$q_e = q_d \otimes q^{-1},$$

with the exact ordering fixed by the project convention.

MATLAB implementation:

```
function q = quatMultiply(q1, q2)
    s1 = q1(1);
    v1 = q1(2:4);
    s2 = q2(1);
    v2 = q2(2:4);

    s = s1*s2 - dot(v1, v2);
    v = s1*v2 + s2*v1 + cross(v1, v2);

    q = [s; v];
end
```

5 Why quatConjugate exists

For

$$q = \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix},$$

the conjugate is

$$q^* = \begin{bmatrix} q_w \\ -q_x \\ -q_y \\ -q_z \end{bmatrix}.$$

For a unit quaternion,

$$q^{-1} = q^*.$$

The inverse represents the opposite rotation. Therefore, if q_{IB} rotates body-frame coordinates into inertial-frame coordinates, then

$$q_{BI} = q_{IB}^{-1} = q_{IB}^*$$

performs the reverse transformation.

This is needed constantly in ADCS. For example, an Earth magnetic-field model may produce a field vector B_I in the inertial frame, while the magnetometer measures the same physical field in the spacecraft body frame B_B . The current attitude is needed to transform between them.

6 Why quatNormalize exists

A valid attitude quaternion must remain unit length:

$$\|q\| = 1.$$

In exact mathematics this property is preserved by the quaternion kinematics. In numerical simulation, floating-point and integration errors can slowly move the quaternion away from unit length.

Normalization restores the constraint:

$$q \leftarrow \frac{q}{\|q\|}.$$

Geometrically, unit quaternions lie on the three-sphere S^3 in four-dimensional space. Numerical drift moves the state slightly away from that surface; normalization projects it back.

A near-zero magnitude check is also essential. The all-zero quaternion does not represent any orientation, and dividing by a near-zero norm would create invalid numerical values. Failing immediately is safer than allowing corrupted attitude data to propagate through the ADCS stack.

7 Why convert a quaternion to a DCM?

Quaternions are excellent for storing and propagating orientation, but many ADCS calculations involve rotating ordinary three-dimensional vectors. A Direction Cosine Matrix (DCM) is convenient for this.

For the convention used here,

$$v_I = C_{IB} v_B.$$

Thus a vector expressed in spacecraft body coordinates can be expressed in inertial coordinates with a matrix multiplication.

For a scalar-first quaternion $q = [q_w, q_x, q_y, q_z]^T$,

$$C_{IB} = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix}.$$

This is useful for transformations such as

$$\text{Sun}_I \leftrightarrow \text{Sun}_B, \quad B_I \leftrightarrow B_B, \quad \text{Earth direction}_I \leftrightarrow \text{Earth direction}_B.$$

8 Why a DCM is 3×3

A spatial rotation maps one three-component vector into another:

$$\mathbf{v}_{new} = C \mathbf{v}_{old}.$$

Therefore $C \in \mathbb{R}^{3 \times 3}$.

A valid rotation matrix satisfies

$$C^T C = I$$

and

$$\det(C) = +1.$$

Orthogonality means the rotation preserves lengths and angles. The determinant requirement distinguishes a proper rotation from a reflection.

9 Why the tests matter

A smooth simulation plot is not proof that the math is correct. Each test checks a physical or mathematical invariant.

Test 1: identity multiplication

The identity quaternion is

$$q_I = [1 \ 0 \ 0 \ 0]^T.$$

It represents zero rotation, so

$$q_I \otimes q = q, \quad q \otimes q_I = q.$$

This is the rotational analogue of multiplying an ordinary number by 1.

Test 2: normalization

If a quaternion has non-unit magnitude, normalization should return a unit quaternion. This validates the projection back onto the valid rotation space.

Test 3: conjugate/inverse property

For a unit quaternion,

$$q \otimes q^* = q_I.$$

This simultaneously checks conjugation, multiplication, and the inverse relationship.

Test 4: identity DCM

The identity quaternion must generate

$$C = I_3,$$

because no rotation means body and inertial axes coincide.

Test 5: DCM orthogonality

We verify

$$CC^T = I.$$

If $v' = Cv$, then

$$\|v'\|^2 = v^T C^T C v = v^T v,$$

so the rotation changes direction but not magnitude.

Test 6: DCM determinant

A physical rotation has

$$\det(C) = +1.$$

An orthogonal matrix with determinant -1 represents a reflection rather than a rigid-body rotation.

Test 7: known 90° rotation

Use

$$q = [\cos 45^\circ \quad 0 \quad 0 \quad \sin 45^\circ]^T$$

and

$$v_B = [1 \quad 0 \quad 0]^T.$$

For the selected convention, a positive 90° rotation about +Z should produce

$$v_I = [0 \quad 1 \quad 0]^T.$$

This test is especially important because it checks the convention as a whole: right-hand rule, scalar-first ordering, Hamilton multiplication, body-to-inertial meaning, DCM formula, and positive rotation direction.

10 How all of this connects to the future ADCS

A typical closed loop eventually looks like this:

Environment → Sensors → Estimator → Guidance → Controller → Actuators → Dynamics.

The quaternion functions support that entire chain:

- `quatMultiply`: combines rotations and forms attitude error.
- `quatConjugate`: produces inverse rotations.
- `quatNormalize`: maintains a valid numerical attitude representation.
- `quatToDCM`: transforms physical vectors between body and inertial frames.

Later, the spacecraft rotational dynamics will propagate angular rate and attitude using

$$J\dot{\omega} + \omega \times J\omega = \tau$$

and

$$\dot{q} = \frac{1}{2}q \otimes \begin{bmatrix} 0 \\ \omega \end{bmatrix}.$$

These quaternion utilities are therefore not throwaway exercises. They are foundational flight-math functions that will be reused by simulation, estimation, guidance, control, and eventually the C++ ADCS implementation.

11 Key takeaway

The purpose of this stage is not merely to “make quaternion code run.” It is to establish a consistent, testable mathematical language for spacecraft orientation. Once the convention and quaternion operations are verified, the next step is to connect them to rigid-body dynamics and create the first physically rotating CubeSat model.